

西安电子科技大学

SoC 微体系结构设计 课程实验报告

实验名称 8 位 SOC 综合设计

计算机科学与技术 学院 2103014 班

姓名 司武强 学号 21009200454

实验日期 2024 年 4 月 25 日

实验地点 E-II 505 实验批次 第一批

成绩

指导教师评语：

指导教师：

____年____月____日

实验报告内容基本要求及参考格式

- 一、实验目的
- 二、实验所用仪器（或实验环境）
- 三、实验基本原理及步骤（或方案设计及理论计算）
- 四、实验数据记录（或仿真及软件设计）
- 五、实验结果分析及回答问题（或测试环境及测试结果）

一、实验目的

熟悉 CPU 基本工作原理及执行顺序，设计完成 8 位 SOC 功能。

二、实验基本原理及内容

- 1) SOC 基本组成单元连接
- 2) 每条指令能够执行
- 3) 设计一段可执行的程序，并转换为相应的指令码，能够正确执行，并将相应的结果显示 在数码管上。

三、实验代码

1. SCO 设计 VHDL 源码

实例化部分：

```
clock_inst:clock port map(clk,nreset,clk1,nclk1,clk2,nclk2,w0,w1,w2,w3);

PC_inst:PC port map(nreset,clk_PC,M_PC,nLD_PC,nPCH,nPCL,PC_in,ROM_PC,DATA_BUS);

ROM_inst:ROM generic map(12,8)port map(clk_ROM,M_ROM,ROM_EN,ROM_PC,DATA_BUS);

uC_inst:uC port map(clk_uC,nreset,M_uA,IR_line(7 downto 2),CMROM_CS,CTR_BUS);

IR_inst:IR port

map(rst=>nreset,clk_IR=>clk_IR,LD_IR1=>LD_IR1,LD_IR2=>LD_IR2,LD_IR3=>LD_IR3,nARen=>nARen,DATA_BUS=>DATA_BUS,IR_out=>IR_line,RS=>RS,RD=>RD,PC_out=>PC_in,AR=>AR(6 downto 0));

RAM_inst:RAM generic map(7,8)port

map(clk_RAM=>clk_RAM,n_reset=>nreset,RAM_CS=>RAM_CS,nRAM_EN=>nRAM_EN,Wr_nRD=>Wr_nRD,AR=>AR,DATA_BUS=>DATA_BUS);

IO_inst:IO port map(clk_IO,nreset,IO_CS,IO_IEN,IO_OEN,RI_EN,RO_EN,data_in,data_out,DATA_BUS);

--SP_inst:SP PORT MAP(clk_SP,nreset,SP_CS,SP_UP,SP_DN,nSP_EN,AR,DATA_BUS);

ALU_inst:ALU port

map(clk,clk_ALU,nreset,m_A,m_B,m_F,nALU_EN,nPSW_EN,C0,S,F_in,regA_out,regB_out,result_out,DATA_BUS,AC,CY,ZN,OV);

RN_inst:RN port map(clk_RN,Rn_CS,nRi_EN,RDRi,WRRi,RS,RD,DATA_BUS);
```

时钟部分：

```
clk_PC<=clk1 and clk2;

clk_ROM<=nclk1 and clk2;

clk_IR<=clk1 and nclk2;

clk_uC<=nclk1 and nclk2;
```

```

clk_RN<=clk1 and nclk2;

clk_ALU<=nclk1 and nclk2;

clk_IO<=clk2;

--clk_IO<=nclk1 and clk2;

clk_RAM<=nclk1 and w1;

clk_SP<=clk1 and clk2 and w1;

```

其他：

```

process (nPCH, nPCL)
begin
    if (nPCH='0') then PC_in(11 downto 8) <= DATA_BUS(3 downto 0);
    elsif (nPCL='0') then PC_in(7 downto 0) <= DATA_BUS;
    end if;
end process;

```

2. 可执行程序一：IO 循环输出流水灯

```

00100100 --IR:24H MOV Ri,#data
00000001 --data:01H
00110100 --IR:34H MOV IO,Ri
00100100 --IR:24H
00000010 --data:02H
00110100 --IR:34H
00100100 --IR:24H
00000100 --data:04H
00110100 --IR:34H
00100100 --IR:24H
00001000 --data:08H
00110100 --IR:34H
00100100 --IR:24H
00010000 --data:10H

```

```

00110100 --IR:34H
00100100 --IR:24H
00100000 --data:20H
00110100 --IR:34H
00100100 --IR:24H
01000000 --data:40H
00110100 --IR:34H
00100100 --IR:24H
10000000 --data:80H
00110100 --IR:34H
00000100 --JMP addr12
00000000 --addr1:00H
00000000 --addr2:00H
00000000

```

3. 可执行程序二：IO 输入加法

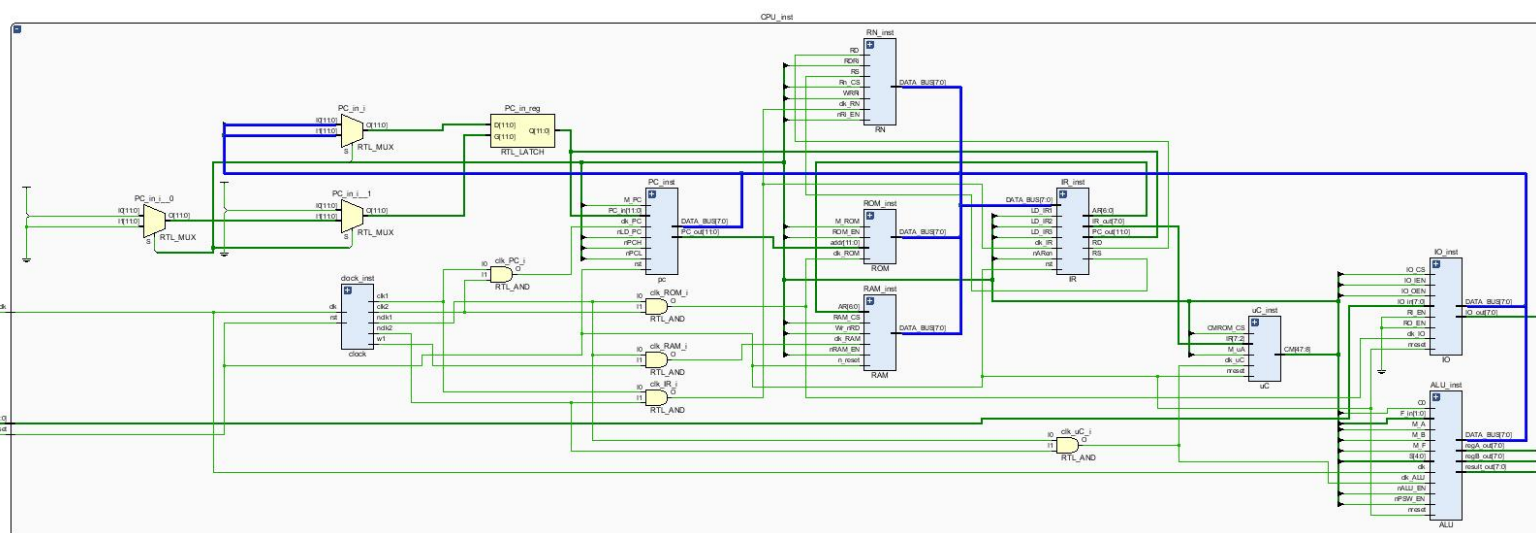
```

00111000 --IR:38H MOV Ri, IO
00111011 --IR:38H MOV Ri, IO
00001000 --IR:08H ADD R0, R1
00000100 --IR:04H JMP addr12
00000000 --addr1:00H
00000000 --addr2:00H

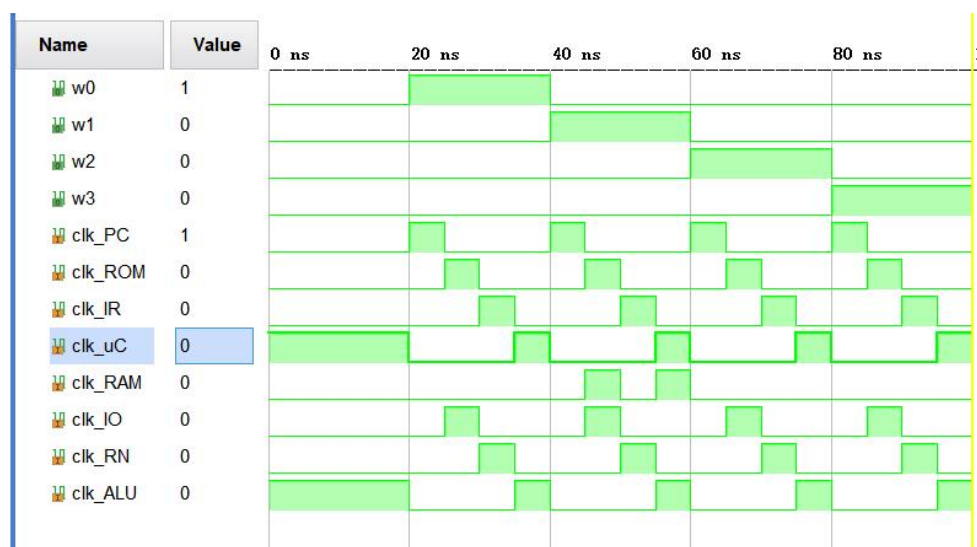
```

四、实验仿真结果

1. 综合结构图

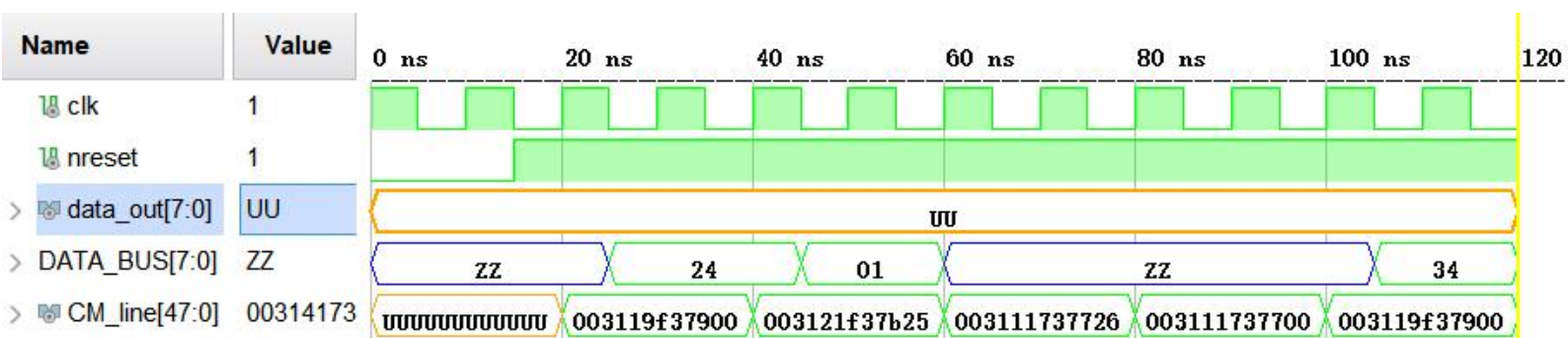


2. 各模块时钟

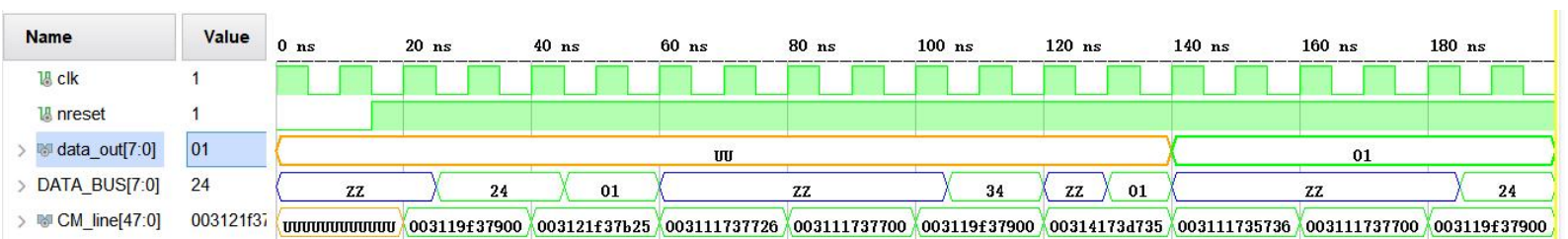


3. 可执行程序一：IO 循环输出流水灯

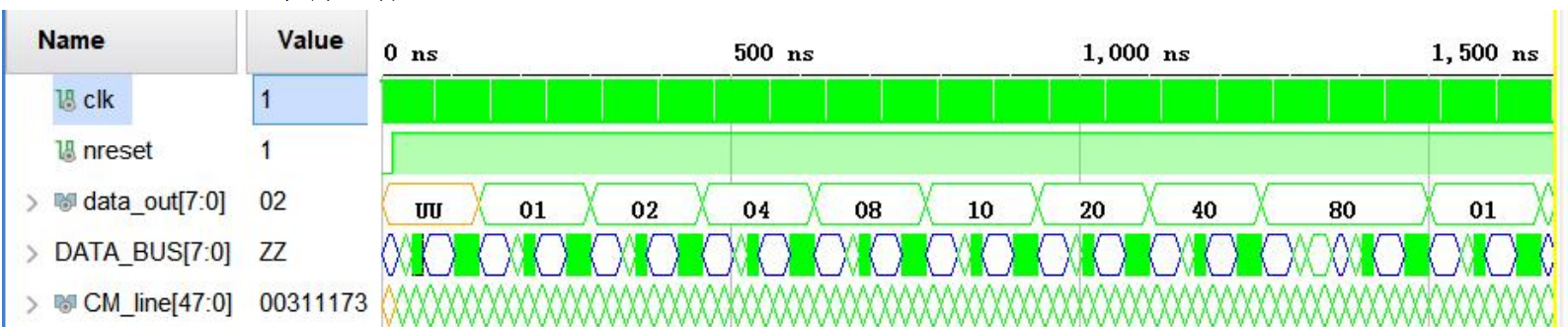
取数 (00100100 --IR:24H MOV Ri, #data 00000001 --data:01H) :



IO 输出 (00110100 --IR:34H MOV IO,Ri) :

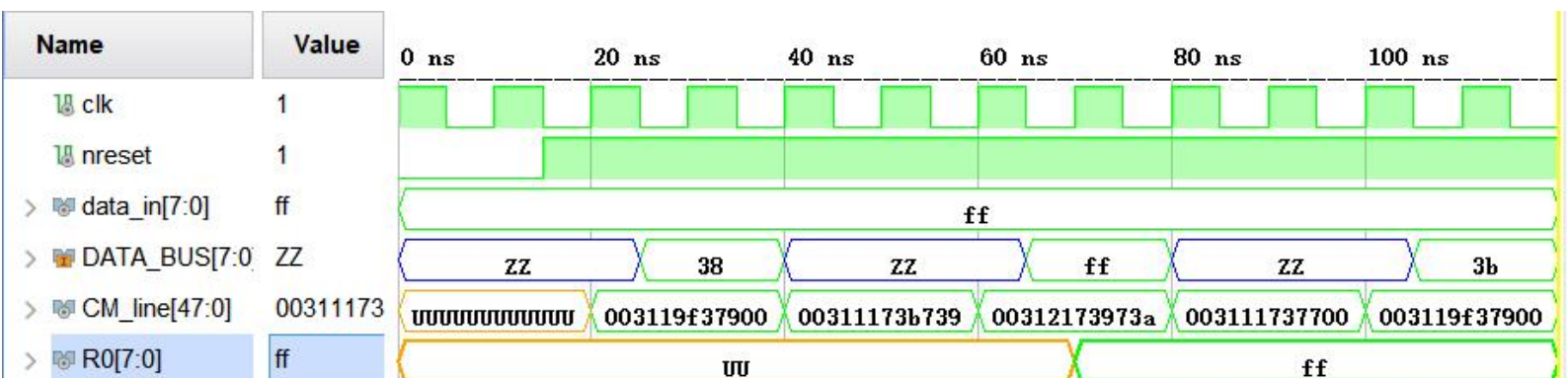


程序整体:

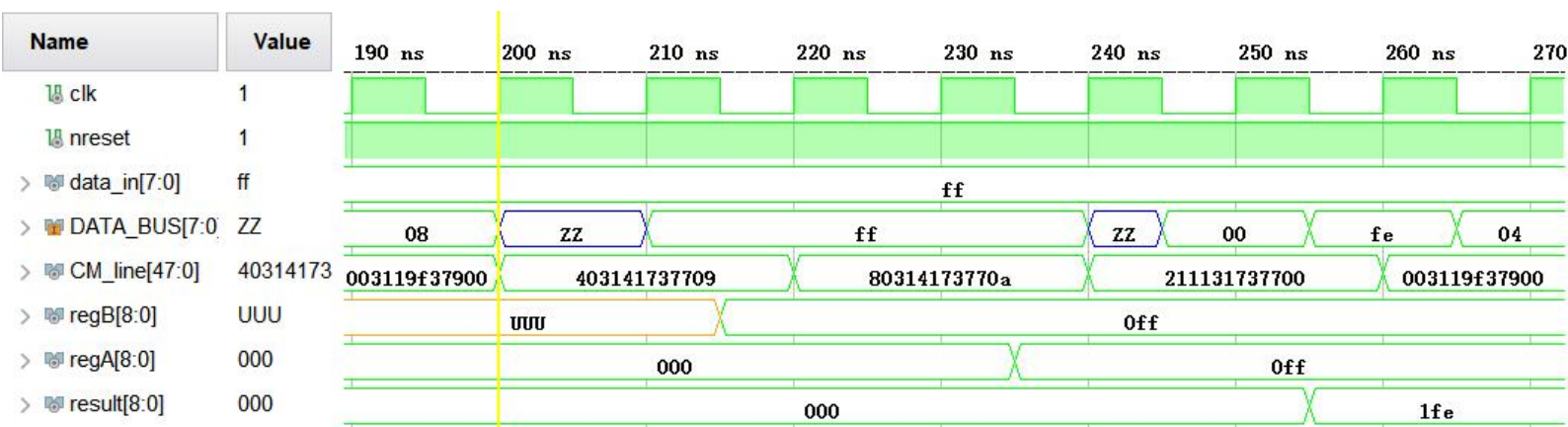


4. 可执行程序二: IO 输入加法

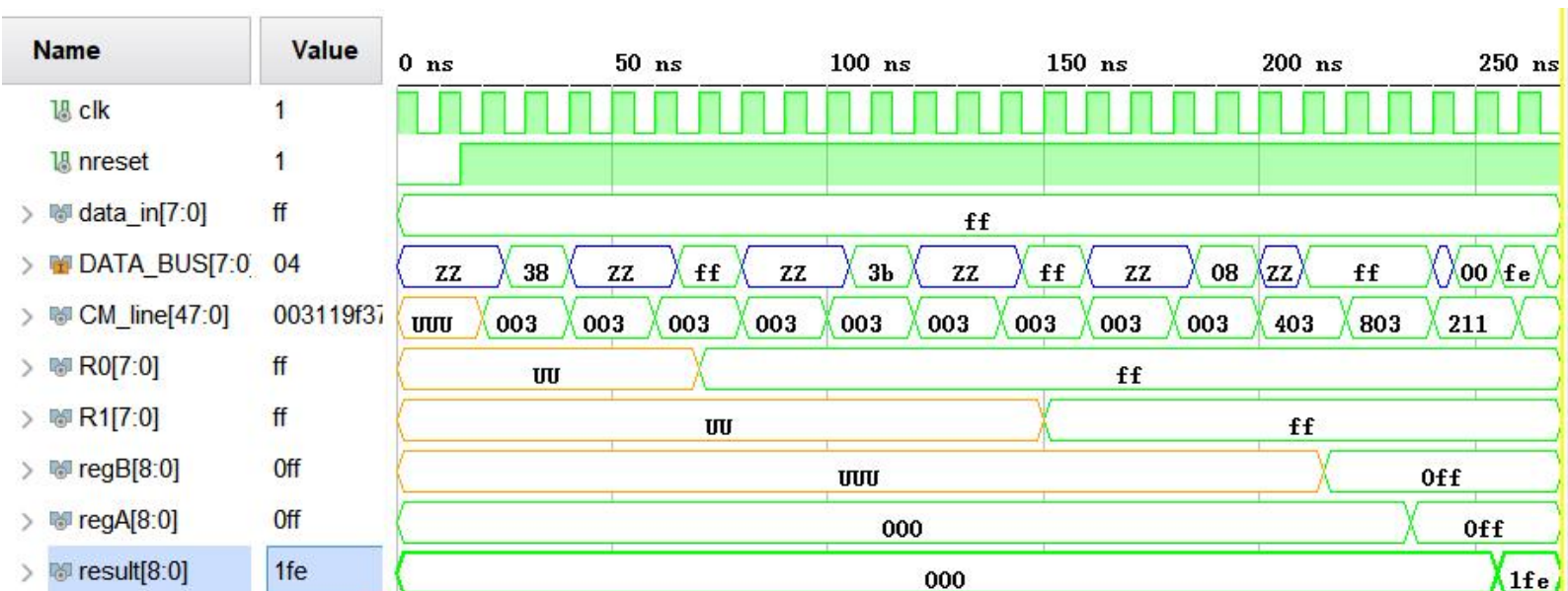
IO 输入 (00111000 --IR:38H MOV Ri,IO) :



加法 (00001000 --IR:08H ADD R0,R1) :



程序整体：



五、实验结果分析及心得体会

1. 实验结果分析

(1) 模块工作顺序分析

各个模块工作的逻辑顺序如上时钟时序图所示，每条指令执行则可分为 $m0 \sim m3$ 四个阶段。在 $m0$ 阶段首先由 PC 发送地址至 ROM，然后 ROM 输出该地址的指令或数据，传送至 IR，IR 经过简单处理后继续传送至 UC，UC 产生各个模块的控制信号。在 $m1 \sim m3$ 阶段，依据微程序产生 3 次控制信号，实现相应指令功能（注意 ALU、RN、IO 时钟不同，后面有相关解释）。

(2) 指令执行过程分析

① 以 LED 流水灯为例：

$m0$ ——取指令：PC→ROM→DATA_BUS→IR→UC，微指令为 0x003119f37900H 注意到 ROM 将地址信息输出到总线，IR 接收并传递地址到 UC，从而执行 24H 地址的微程序。

MOV Ri, #data:

m1~3——执行 24H MOV Ri, #data: 在 m1 将 ROM 中的立即数输出至 DATA_BUS, 在 m2 传输至 Ri 之中, 在 m3 阶段 UC 地址回到 00H 准备进入下一条指令, 即取值公操作。

MOV IO, Ri:

m1~3 —— 执行 34H MOV IO, Ri: 在 m1 将 Ri->DATA_BUS, 在 m2, DATA_BUS->REG->IO_OUT (REG 是 IO 内暂存器, 由于 UC 控制信号不包含 RI_EN 和 RO_EN, 一直使能, 即 DATA_BUS<->IO 直接完成), 在 m3 阶段 UC 地址回到 00H 准备进入下一条指令, 即取值公操作。

JMP addr12:

m1~3——执行 04H JMP addr12: 在 m1、m2 依次将立即数加载至 IR 中, 对应 LD_IR2 和 LD_IR3 分别有效, IR 将新地址传送至 PC。

② 以 ADD 加法为例:

MOV Ri, IO:

m1~3——执行 38H MOV Ri, IO: 在 m1 使能 IO 相关信号使 IO_IN->REG, 在 m2 时, REG->DATA_BUS->Ri (IO 的时钟先于 RN, 使数据在 DATA_BUS 刚好能传送至 RN)

ADD R0, R1:

m1~3——执行 08H ADD R0, R1: 由于无法在计算出结果的同时将结果通过数据总线 DATA_BUS 传送至 RN, 所以在一条指令中, 我的 ADD 指令只能实现计算功能, 计算结果由 ALU 另行输出至 result 总线。在 m1 和 m2 分别实现 RN->DATA_BUS->regA/B (其中 regA/B 为 ALU 内部缓冲器), 在 m3 实现计算和 UC 地址回到 00H 操作。

2. 心得体会

(1) SOC 模块集成设计经验

在集成各个组成模块时, 为避免总线冲突以及方便调试, 采用各个模块依次添加测试的方法。

首先测试 PC、ROM、IR、UC 组成的系统, 要求取指令时 4 个模块的运行顺序为: PC 发送 ROM 地址->ROM 输出指令/数据->IR 传送指令->UC 解析执行微程序。随后依次加入 IO、RN、ALU、RAM 模块进行调试 (按照指令执行逻辑顺序)。

(2) SOC 集成关键

在集成各个模块时, 关键在于:

① 清楚模块工作逻辑顺序

需清楚知晓各个模块工作先后关系, 理清数据/控制/地址信号流的传递顺序, 以避免读写顺序出错, 出现写未读到的情况。

② 避免总线冲突

主要是 DATA_BUS 数据总线使用的原子性, 各模块严格按照时钟时序进行输出, 在允许输出时输入/出数据到数据总线, 其他时刻不要忘记将数据总线

000000000011000100010101111100110111101100000101	--04H JMP addr12
000000000011000100010011111100110111101100000110	--05H
000000000011000100010001101100110111011100000000	--06H
000	--07H
010000000011000101000001011100110111011100001001	--08H ADD Ri,Rj
100000000011000101000001011100110111011100001010	--09H
001000010001000100110001011100110111011100000000	--0AH
000000000001000100100001111100110111011100000000	--0BH
...	
000000000011000100100001111100110111101100100101	--24H MOV Ri,#data
000000000011000100010001011100110111011100100110	--25H
000000000011000100010001011100110111011100000000	--26H
000	--27H
000000000011000101100001011100110111011100101001	--28H
000000000011000101100001011100110111011100101010	--29H
000000000011000100010001011100110111011100000000	--2AH
000	--2BH
000000000011000100010010111100110111101100101101	--2CH
000000000011010000100000011100110111011100101110	--2DH
000000000011000100010001011100110111011100000000	--2EH
000	--2FH
000000000011000100010010111100110111101100110001	--30H
000000000011011101000000011100110111011100110010	--31H
000000000011000100010001011100110111011100000000	--32H
000	--33H
000000000011000101000001011100111101011100110101	--34H MOV IO,Ri
000000000011000100010001011100110101011100110110	--35H
000000000011000100010001011100110111011100000000	--36H
000	--37H
000000000011000100010001011100111011011100111001	--38H MOV Ri,IO
000000000011000100100001011100111001011100111010	--39H
000000000011000100010001011100110111011100000000	--3AH
000	--3BH

2. 相较于课程资料微程序的改动

修改微指令 34H: IO_CS 位

修改微指令 38H: IO_CS 位

修改微指令 24H: Ri_EN 位

修改微指令 8AH: M_PC 位

修改微指令 65H: RDRi 位

ADD 微程序增加一条微指令